

04

PR 리뷰 · 문서 자동화

GitHub · Excel · Word · PowerPoint

차성재 · Agentic AI PM / AI 겸임교수

4주차 학습 경로

앞선 수업	오늘의 확장	다음 주
1·2주차 도구 호출·회의 기록	검증 가능한 입력과 파일	업무 서비스 통합
3주차 코드 리뷰 Agent	PR 연동·테스트·사람 확인	개인 기능 개선
문서 활용 경험	수식·템플릿·생성 코드	3시간 개인 미니 프로젝트

하루 시간표

수업 시간 (쉬는 시간)	차시	함께 챙길 내용
09:00-12:00 (11:30-12:00)	1·2·3차시	PR 대상·리뷰·테스트
12:00-13:00	점심시간	13시 수업 시작
13:00-15:00 (14:40-15:00)	4·5차시	WBS·자동 채점
15:00-18:00 (17:30-18:00)	6·7·8차시	Word·PPT·통합 마지막 30분: 휴식·Q&A

실습 환경

필수 준비	확인	선택 연결
Python 3.12·VS Code	Notebook 커널 선택	Codex 로컬 프로젝트
제공 저장소 또는 ZIP	requirements-day4.txt 설치	Claude Code 로컬 폴더
파일을 열 수 있는 도구	XLSX·DOCX·PPTX·브라우저	개인 GitHub·Google 계정
합성 데이터	실제 고객·성적·키 미포함	계정 제한 시 로컬 경로

Notebook 시작

```
python -m pip install -r requirements-day4.txt  
python -m jupyter lab
```

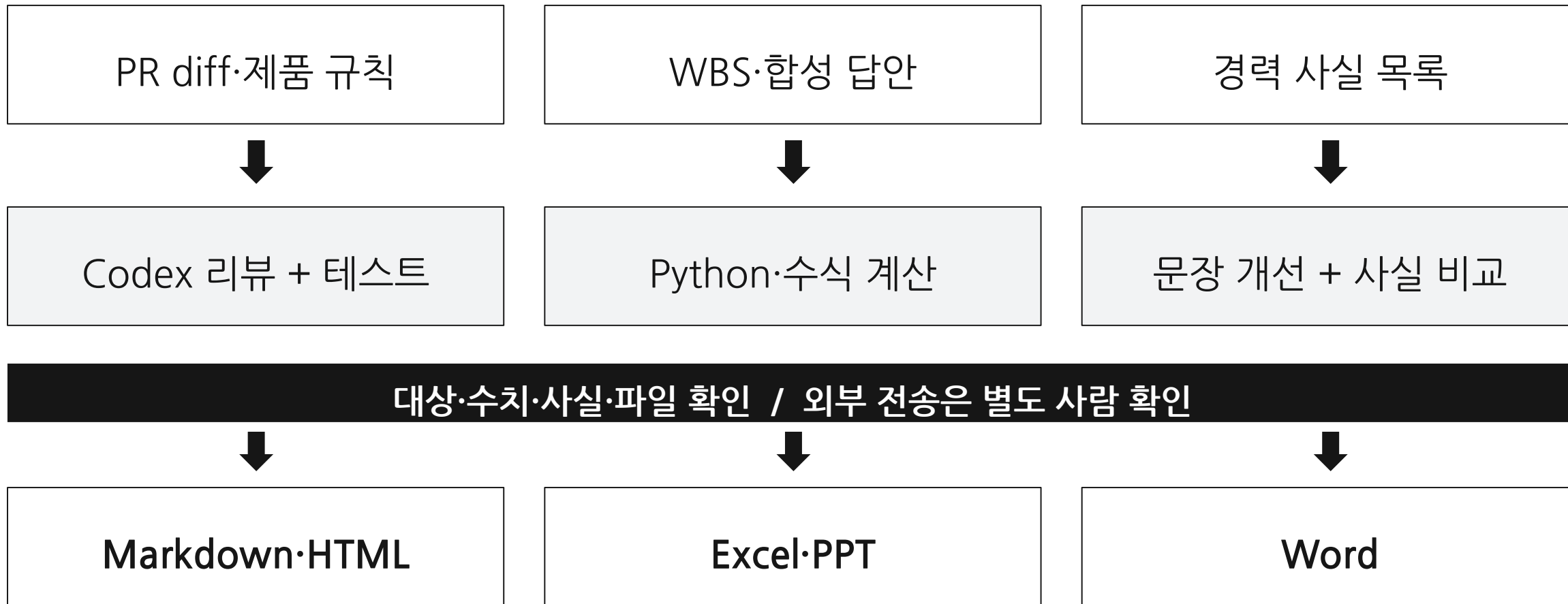
materials/day4/day4_pr_document_automation.ipynb

추가 코드는 해당 차시에 새 코드 셀로 실행

도구별 작업 공간

도구	작업 방식	이번 수업의 용도
Codex 로컬 프로젝트	폴더·코드·터미널·diff	함수 수정과 테스트 실행
Claude Code 로컬 환경	허용한 폴더의 파일·명령	동일 과제의 다른 구현
ChatGPT·Claude 대화	첨부한 파일·사용 가능 도구	문장·문서 초안과 비교
Python·Node 코드	정해진 입력과 규칙	계산·반복 생성·자동 검사

입력에서 결과 파일까지



4주차 1차시

09:00-09:50

PR 대상과 변경 코드

저장소·PR·commit 구분과 변경 코드 검증

이론 12분 · 시연 8분 · 작업 25분 · 확인 5분

pr.json · changes.diff · review.html

리뷰 결과의 목적지

같은 코드라도 저장소와 PR이 다르면 다른 작업

commit SHA: 리뷰가 기준으로 삼은 코드 버전

좋은 리뷰 문장보다 먼저 확인할 대상 정보

PR 구성

항목	예시	의미
Repository	training-example/checkout-demo	합성 연습 저장소
PR number	42	변경 묶음의 번호
head SHA	40자리 commit 식별값	리뷰 대상 코드 버전
diff	checkout.py 변경 줄	추가·삭제된 코드

AI와 코드의 Role

작업	AI의 Role	코드의 Role
변경 코드 이해	사용자 영향 추론	경로·줄 번호 검사
리뷰 내용	설명과 수정 제안	형식·대상 일치 검사
승인 이후	판단 근거 보조	버전·중복·권한 검사

Diff 읽기

```
- return price * quantity  
+ return price * quantity - coupon
```

- 삭제한 코드 / + 추가한 코드
새 쿠폰 인자와 음수 결제 가능성

입력 검증의 순서

먼저 확인	그다음 확인	문제 발생 시
저장소·PR 번호	예상한 대상과 일치	다른 PR 처리 중단
SHA 40자리 형식	최신 head SHA와 일치	새 diff 수집
파일 경로	작업 폴더 안쪽	접근 차단
변경 줄	리뷰 줄의 실제 존재	의견 수정 요청

PR 리뷰 작업대

```
-def checkout_total(price: int, quantity: int) -> int:
-    """상품 가격 × 수량."""
-    return price * quantity
+def checkout_total(price: int, quantity: int, coupon: int = 0) -> int:
+    """상품 가격 × 수량 - 쿠폰. 잘못된 입력과 초과 쿠폰을 확인하세요."""
+    return price * quantity - coupon
```

합성 PR의 함수 변경 확대 · 로컬 화면 · GitHub 게시 없음

합성 PR 로딩

```
from labs.day4.pr_review_lab import (  
    load_fixture, validate_snapshot  
)  
snapshot = load_fixture()  
validate_snapshot(snapshot,  
    expected_repo=snapshot["repo"], expected_pr=42)
```

코드: labs/day4/pr_review_lab/
Notebook: 1차시

Notebook 실행 위치

```
print("Python:", sys.executable)
print("프로젝트:", ROOT)
print("실행 위치:", Path.cwd())
print((ROOT / "requirements-day4.txt").is_file())
```

Notebook 준비 셀 실행 후 새 코드 셀에 입력
마지막 값 True · ROOT 아래 labs 폴더 확인

대상 정보 확인

- 01 Notebook 1차시에서 합성 PR 로딩
- 02 저장소·번호·SHA를 각각 출력
- 03 changes.diff와 checkout.py 변경 줄 비교

labs/day4/pr_review_lab/fixtures/pr.json

세 값의 의미를 구분하고 해당 변경 파일을 열기

PR 번호 검사의 수정

```
def my_target_matches(pr, repo, number):  
    return (  
        pr["repo"].lower() == repo.lower()  
        and pr["number"] == number  
    )  
assert my_target_matches(snapshot, snapshot["repo"], 42)  
assert not my_target_matches(snapshot, snapshot["repo"], 43)
```

and 행 제거 시 PR 43 검사 실패
조건 복구 후 같은 셀의 두 assert 통과

잘못된 대상의 차단

- 01 deepcopy로 snapshot 복사
- 02 repo를 another/repository로 변경
- 03 expected_repo를 원본으로 주고 검증 실행

Notebook · 1차시 실패 입력

REPO_MISMATCH 확인 · 원본 fixture 보존

다른 저장소의 차단

```
from copy import deepcopy
broken = deepcopy(snapshot)
broken["repo"] = "another/repository"
validate_snapshot(broken,
                  expected_repo=snapshot["repo"], expected_pr=42)
```

예상 오류: REPO_MISMATCH
Notebook의 예외 처리 함수로 확인

SHA 입력의 오류 복구

```
bad = deepcopy(snapshot)
bad["head_sha"] = "최신버전"
target = dict(expected_repo=snapshot["repo"], expected_pr=42)
try:
    validate_snapshot(bad, **target)
except LabError as error:
    print(error.code)
bad["head_sha"] = snapshot["head_sha"]
assert validate_snapshot(bad, **target) is bad
```

INVALID_SHA 출력 후 마지막 assert 통과
형식 복구 실험 · 최신 버전 조회는 별도

로컬 작업 폴더 생성

- 01 prepare_exercise로 새 폴더 생성
- 02 checkout.py와 test_checkout.py 열기
- 03 check_exercise로 현재 테스트 실행

Notebook에 출력된 EXERCISE 경로

4 failed, 1 passed · 기존 파일 덮어쓰기 없음

실습 파일의 역할

파일	열어서 확인할 내용	수정 기준
checkout.py	checkout_total 함수 실제 결제 계산	기능 수정 대상
test_checkout.py	입력과 assert 기대값 ValueError 검사	기존 검사 유지 새 검사 추가
reference_solution.py	참고 구현 비교용 정답 코드	필요할 때 비교 자동 복사 기본 꺼짐
checkout_before.py	쿠폰 추가 이전 코드 변경 배경	읽기 전용 비교

생성 셀의 재실행 오류

- 01 실습 폴더 생성 셀을 같은 RUN에서 다시 실행
- 02 EXERCISE_EXISTS와 기존 파일 보존 확인
- 03 생성 셀을 건너뛰고 현재 EXERCISE 경로 확인
- 04 아래 check_exercise 셀에서 기존 수정본 검사

Notebook 1차시 · prepare_exercise / check_exercise

기존 코드 유지 · 검사 셀만 재실행

입력과 실행의 구분

입력	취급	기본 정책
PR 제목·설명	외부 사람이 쓴 자료	명령으로 실행하지 않음
PR 변경 코드	검토할 자료	자동 checkout·실행 금지
수업용 checkout.py	확인된 합성 코드	로컬 테스트 실행
계정 키·고객 파일	실습 대상 아님	읽기·캡처·커밋 제외

1차시 확인

확인 항목	확인 방법
대상 구분	저장소·PR·SHA를 각각 설명
실제 파일	checkout.py와 test_checkout.py 열기
실패 처리	다른 repo 입력의 차단 확인
테스트 출발점	4 failed, 1 passed 출력 보관

API 응답의 누락

상황	피해야 할 처리	필요한 처리
binary 변경	없는 diff를 정상으로 간주	검토 불가 표시
페이지 수 초과	앞부분만 보고 완료 표시	범위 초과 중단
읽는 중 새 commit	서로 다른 버전 혼합	SHA 재확인

Context의 우선순위

제품 규칙과 변경 목적

관련 함수·테스트·호출 위치

리뷰 범위를 넘는 자료는 제외

4주차 2차시

09:50-10:40

리뷰 초안과 코드 수정

Codex 리뷰 요청, 결제 코드 수정, 게시 전 확인

이론 12분 · 시연 8분 · 작업 25분 · 확인 5분

checkout.py · test_checkout.py · review.md

리뷰 의견의 구성

항목	초과 쿠폰 예시
위치	checkout.py:6
사용자 영향	결제액이 -4,000원으로 계산
재현 입력	가격 1,000 · 수량 1 · 쿠폰 5,000
최소 수정	입력 검사 후 결제액 0원 하한
검증	초과 쿠폰과 기존 정상 케이스 테스트

그럴듯한 의견과 유용한 의견

비교	모호한 의견	실행 가능한 의견
문제	예외 처리가 부족함	수량 0이 주문으로 처리됨
수정	클린 코드 적용	quantity < 1 검사 추가
검증	잘 동작하는지 확인	ValueError 테스트 통과

추론 결과의 확인

모델의 설명은 가설과 제안

재현 테스트는 관찰 가능한 증거

제품 규칙은 사람이 정한 기준

리뷰 요청의 Context

필수 자료	이번 파일	선택 이유
업무 규칙	review_policy.md	무엇이 버그인지 판단
구현 코드	checkout.py	어디를 바꿀지 확인
테스트	test_checkout.py	기존 동작 보존
금지 행동	수정·게시 전 확인	범위 확장 방지

Codex 리뷰 요청

대화 입력 예시

review_policy.md와 내 실습 폴더의 checkout.py, test_checkout.py를 읽어 줘. 실패 입력과 사용자 영향을 먼저 설명해 줘. 파일 수정과 GitHub 게시는 아직 하지 마. 의견은 위치·재현 조건·최소 수정·관련 테스트 순서로 써 줘.

확인할 내용

재현 입력 · 수정 범위 · 실행 여부

실제 Codex 리뷰 응답

사람 확인: 응답의 checkout.py:5는 docstring 위치입니다.
실제 계산식은 6행입니다. 아래 원문은 수정하지 않았습니다.

리뷰 결과

1. ****중요도: 높음 — 초과 쿠폰으로 결제액이 음수가 됨****

- ****파일·줄:**** `fixtures/checkout.py:5`
- ****재현 입력:**** `checkout\_total(1000, 1, 5000)` → 현재 `-4000`
- ****사용자 영향:**** 결제액은 0 이상이어야 하지만 음수 금액이 반환되어 환불·정산·주문 처리 오류로 이어질 수 있습니다.
- ****최소 수정:**** 유효성 검사 후 계산 결과를 `max(0, price \* quantity - coupon)`으로 제한합니다.
- ****기존 테스트:**** `test\_coupon\_cannot\_make\_total\_negative`

실제 CLI 응답 일부 확대 · Desktop 대화 UI 아님

모델 의견과 코드 근거

항목	실제 모델 응답	사람 확인
초과 쿠폰	-4,000원 가능	계산식과 일치
입력 검사	수량 0·음수 거절	제품 규칙과 일치
문제 위치	checkout.py:5	계산식은 6행
실행 여부	테스트 미실행	별도 pytest 필요

초과 쿠폰의 재현

```
from checkout import checkout_total
print(checkout_total(1000, 1, 5000))
# 수정 전: -4000
# 요구사항: 0
```

실습 폴더 안에서 import
정상 가격·수량 계산은 유지

결제 입력 규칙

입력	허용	거절
가격	0 이상의 값	음수 가격
수량	1 이상의 값	0 또는 음수
쿠폰	0 이상의 값	음수 쿠폰
최종 결제액	0원 이상	음수 결과

실패 테스트 읽기

- 01 check_exercise 실행
- 02 assert의 실제값과 기대값 비교
- 03 실패한 입력을 제품 규칙과 연결

내 실습 폴더/test_checkout.py

실패 네 건의 원인 구분

pytest 메시지의 구분

표시	뜻	다음 수정
<code>assert -4000 == 0</code>	실제 계산값과 기대값 불일치	checkout.py 최종 금액 확인
<code>DID NOT RAISE</code>	거절해야 할 입력을 함수가 그대로 처리	checkout.py 입력 검사 추가
<code>IndentationError</code>	Python 들여쓰기 오류 기능 검사 전 중단	같은 함수 안에서 공백 깊이 정렬
<code>ModuleNotFoundError</code>	실행 Python에서 모듈을 찾지 못함	커널·설치·경로 확인 기대값 수정 금지

결제 코드 수정

01 가격·수량·쿠폰 입력 검사 추가

02 음수 결제액을 0으로 제한

03 같은 테스트 명령으로 재실행

내 실습 폴더/checkout.py

5 passed · 테스트 기대값 변경 금지

최소 수정 예시

```
if price < 0 or quantity < 1 or coupon < 0:  
    raise ValueError("INVALID_CHECKOUT_INPUT")  
return max(0, price * quantity - coupon)
```

기존 함수 안의 수정

정상 입력 + 초과 쿠폰 + 잘못된 입력

수정 파일과 저장 상태

- 01 Notebook이 출력한 EXERCISE/checkout.py 열기
- 02 checkout_total 함수 안에 검사·반환식 수정
- 03 Ctrl+S 또는 ⌘S로 저장하고 수정 표시 확인
- 04 current_test 셀만 다시 실행해 새 결과 확인

내 실습 폴더/checkout.py · Notebook current_test 셀

기본 제공 5개 테스트 모두 통과

정상 계산의 회귀 검사

```
command = [  
    sys.executable, "-m", "pytest", "-q",  
    "test_checkout.py", "-k", "normal_coupon",  
]  
normal = subprocess.run(  
    command, cwd=EXERCISE, capture_output=True, text=True  
)  
print(normal.stdout)  
assert normal.returncode == 0
```

기본 샘플: 1 passed, 4 deselected

회귀 검사: 수정 전 정상 기능의 유지 확인

수정 Diff의 재검토

대화 입력 예시

내가 수정한 checkout.py의 diff를 리뷰해 줘. 정상 쿠폰 계산이 유지되는지, 잘못된 입력이 거절되는지, 테스트를 약하게 바꾸지 않았는지 확인해 줘. 불필요한 라이브러리와 추상화는 추가하지 마.

확인할 내용

기존 기능 · 입력 오류 · 테스트 보존

수정 전후의 파일 비교

```
from difflib import unified_diff
source = ROOT / "labs/day4/pr_review_lab/fixtures/checkout.py"
old = source.read_text(encoding="utf-8").splitlines()
new = (EXERCISE / "checkout.py").read_text(encoding="utf-8")
diff = unified_diff(
    old, new.splitlines(), fromfile="before", tofile="my-checkout",
    lineterm=""
)
print("\n".join(diff))
```

+ 입력 검사 · + 0원 하한의 추가 확인
로컬 파일 비교 · GitHub PR 갱신과 별개

승인의 범위

확인 대상	고정할 값
어디에	저장소와 PR 번호
어떤 코드에	head SHA
어떤 내용을	리뷰 본문의 해시
누가 확인했는지	검토자 식별값

게시 전 미리보기

```
approval = approve_preview(  
    snapshot, review,  
    approved=True, reviewer="student")  
preview = preview_publication(  
    snapshot, review, approval,  
    current_head_sha=snapshot["head_sha"])
```

본문을 읽은 뒤 approved=True 선택
PREVIEW_ONLY · 원격 게시 없음

수정 결과의 HTML 저장

```
current_test = check_exercise(ROOT, EXERCISE)
folder = RUN / "pr" / ("report-" + uuid.uuid4().hex[:6])
files = render_outputs(
    ROOT, folder, snapshot, review,
    test_result=current_test
)
file_link(files["review.html"])
```

새 HTML의 현재 로컬 테스트 결과 확인
원본 PR 리뷰는 유지 · 게시 상태는 확인 대기

2차시 확인

확인 항목	확인 방법
수정 결과	5 passed · 변경 diff 확인
리뷰 문서	재현 조건과 최소 수정 포함
게시 구분	preview는 전송 기록이 아님
사람 검토	초안의 내용과 대상을 직접 확인

리뷰 품질 평가

기준	좋은 상태	나쁜 상태
정확성	재현 가능한 결함	없는 문제의 지적
관련성	이번 변경의 영향	전체 구조 재작성
실행 가능성	작은 수정과 테스트	추상적 개선 요구
노이즈	중요한 의견만 유지	유사 의견의 반복

금액 처리의 확장

정수 단위 또는 Decimal의 명시

반올림·세금·할인 순서의 제품 규칙

데이터형과 최대 금액의 추가 테스트

4주차 3차시

10:40-11:30

테스트와 GitHub 연동

실패 테스트 복구와 CI·재실행 안전 설계

이론 12분 · 시연 8분 · 작업 25분 · 확인 5분

workflow.yml · publication_preview.json

자동 검사와 AI 리뷰

구분	자동 검사·CI	AI 리뷰
질문	정한 규칙을 통과하는가	놓친 위험이 있는가
결과	명확한 pass/fail	설명과 판단 보조
반복 실행	같은 조건의 재현성	출력의 변동 가능성
최종 결정	테스트 증거	사람의 merge 판단

GitHub 연동 구조

순서	작업	이번 기본 실습
1	PR 정보·diff 읽기	합성 PR / 선택 공개 PR
2	리뷰와 테스트 준비	로컬 파일·pytest
3	게시 내용·대상 확인	사람 확인 미리보기
4	외부 전달	기본 경로에서 미실행

재실행의 두 가지 위험

리뷰 이후 코드가 바뀌는 상황

네트워크 재시도로 같은 의견을 두 번 쓰는 상황

새 버전 확인과 중복 확인의 별도 처리

통과 테스트의 확인

```
result = check_exercise(ROOT, EXERCISE)
print(result["status"])
print(result["output"])
```

수정 완료 후: PASSED / 5 passed
EXERCISE는 Notebook이 만든 실습 폴더

참고 구현의 테스트 결과

```
reference_check = check_exercise(ROOT, RUN / "pr/reference-only")
print(reference_check["output"])
# 실제 참고 구현 검증: 5 passed
```

내 작업 폴더가 아닌 별도 참고 구현의 결과
Notebook의 참고 구현 준비 셀 실행 후 비교

Actions의 최소 구성

```
on: [pull_request]
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      # 제공 템플릿의 checkout·setup 단계
      - run: python -m pytest -q exercise
```

전체 파일: templates/workflow.yml

축약 설명 · 실행은 완전한 템플릿 사용

권한의 최소 범위

필요 기능	필요 권한	이번 선택
코드 읽기	contents: read	허용
테스트 실행	로컬·CI runner	허용
PR 댓글 쓰기	추가 쓰기 권한	기본 제외
자동 merge	저장소 변경 권한	금지

테스트 한 건 추가

- 01 기존 정상 쿠폰 테스트 확인
- 02 본인이 고른 입력 한 가지 추가
- 03 실패 조건과 통과 조건을 비교

내 실습 폴더/test_checkout.py

단순 실행 외에 테스트 코드 직접 작성

쿠폰 경계값 테스트

```
@pytest.mark.parametrize("coupon, expected", [
    (0, 1000),
    (1000, 0),
])
def test_zero_or_exact_coupon(coupon, expected):
    assert checkout_total(1000, 1, coupon) == expected
```

test_checkout.py 맨 아래에 추가 후 저장
쿠폰 없음과 총액만큼의 쿠폰: 두 경우 검사

추가 테스트의 선택 실행

```
command = [  
    sys.executable, "-m", "pytest", "-q",  
    "test_checkout.py", "-k", "zero_or_exact_coupon",  
]  
extra = subprocess.run(  
    command, cwd=EXERCISE, capture_output=True, text=True  
)  
print(extra.stdout)  
assert extra.returncode == 0
```

Notebook 새 셀: 2 passed, 5 deselected
그다음 check_exercise 전체 결과 7 passed

CI 파일 구성

01 workflow.yml을 별도 연습 폴더로 복사

02 trigger·permissions·pytest 단계 찾기

03 내 exercise 경로와 실행 명령 확인

labs/day4/pr_review_lab/templates/workflow.yml

전체 YAML과 실제 폴더 경로의 일치

개인 저장소의 파일 배치

복사할 파일	개인 저장소의 위치	연결되는 설정
내 checkout.py	exercise/checkout.py	검사할 기능 코드
내 test_checkout.py	exercise/test_checkout.py	pytest가 읽는 테스트 파일
제공 workflow.yml	.github/workflows/day4-checkout-tests.yml	working-directory: exercise
로컬 실행 위치	터미널에서 exercise 폴더	python -m pytest -q test_checkout.py

오래된 코드의 차단

```
preview_publication(  
    snapshot, review, simulated_approval,  
    current_head_sha="3" * 40  
)
```

예상 결과: STALE_HEAD_SHA

Notebook 3차시 expected_failure 셀 사용

중복 리뷰의 차단

01 같은 리뷰 표식으로 이력 구성

02 Notebook 중복 검사 셀 실행

03 새 리뷰와 재전송 상황 비교

Notebook · 3차시 안전 검사

DUPLICATE_REVIEW · 두 번째 전송 없음

리뷰 수정과 이전 승인

```
findings = deepcopy(review["findings"])
findings[0]["suggestion"] += " 정상 입력도 함께 검사합니다."
revised = build_review(snapshot, findings, provider="human")
expected_failure("이전 승인", lambda: preview_publication(
    snapshot, revised, simulated_approval,
    current_head_sha=snapshot["head_sha"]),
    "APPROVAL_MISMATCH")
```

같은 PR·같은 commit이어도 내용 변경은 재확인
테스트용 승인 객체 · 원격 전송 없음

수정 리뷰의 재확인

- 01 `display(Markdown(revised["markdown"]))`로 새 문서 확인
- 02 추가한 제안·위치·재현 조건을 원본과 대조
- 03 실제 검토 후 승인 예시의 `review`를 `revised`로 변경
- 04 새 `approval`과 `revised`로 미리보기 셀 재실행

Notebook 3차시 · `revised` / `approve_preview`

사람 확인 후 `PREVIEW_ONLY` · GitHub 게시 없음

GitHub 연결 요청

대화 입력 예시

내 개인 연습 저장소에서만 진행해 줘. 변경 파일과 테스트 결과를 정리하고 PR 설명 초안을 만들어 줘. push와 PR 생성은 대상 저장소·브랜치·공개 범위를 보여준 뒤 내 확인을 기다려 줘. merge는 하지 마.

확인할 내용

저장소 · 브랜치 · diff · 테스트 결과

개인 PR의 확인 위치

화면	확인 내용
Files changed	의도한 파일만 변경
Checks	같은 테스트 명령의 통과
Conversation	목적·수정 범위·주의점
Merge 전	사람의 최종 확인

3차시 확인

확인 항목	확인 방법
테스트	내가 추가한 테스트 포함 통과
CI 파일	경로·권한·실행 명령 확인
버전 변경	STALE_HEAD_SHA 차단
재실행	DUPLICATE_REVIEW 차단

운영 확장 · 네트워크 오류

응답	처리 방향
401·403	인증·권한·요청 제한 확인
429	제한 확인 후 정해진 횟수만 대기
5xx·timeout	읽기는 제한 재시도, 쓰기는 이력 확인
알 수 없는 전송 결과	재전송 전에 실제 게시 이력 조회

CI의 공급망 위험

Action 버전을 전체 commit SHA로 고정

PR에서 온 스크립트와 secret의 분리

검사 도구 업데이트도 변경 이력으로 관리

쉬는 시간 · 점심시간

11:30-13:00

쉬는 시간 11:30-12:00

점심시간 12:00-13:00

4주차 4차시

13:00-13:50

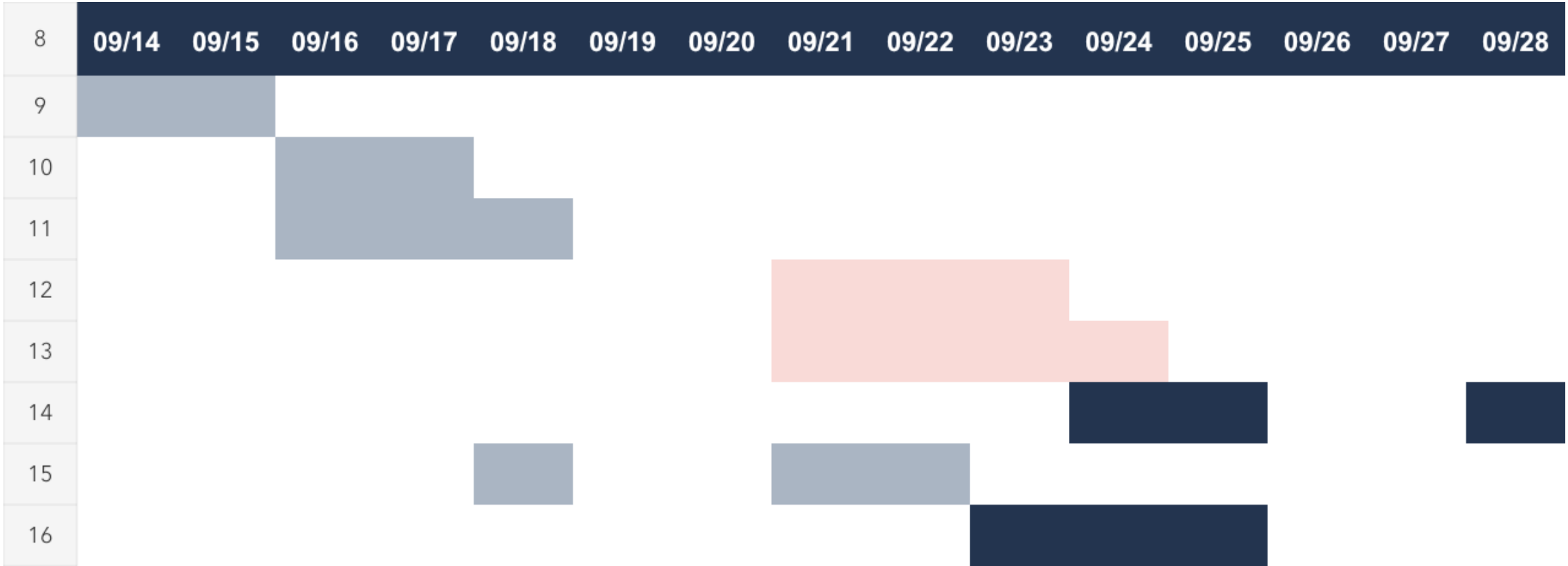
Excel WBS와 간트 차트

입력·수식·조건부서식으로 작동하는 일정표

이론 12분 · 시연 8분 · 작업 25분 · 확인 5분

wbs_tasks.json · WBS_Gantt.xlsx

일정 데이터와 간트 차트



9/14~9/28 구간 확대 · 첫 8개 업무 · 전체 일정은 XLSX

WBS 연습 파일 열기

- 01 제공 파일을 복사한 뒤 Excel 또는 LibreOffice로 열기
- 02 아래쪽 WBS 탭 선택, 왼쪽 위 이름 상자에 F12 입력
- 03 같은 행의 A12가 W04인지, F12가 80%인지 확인
- 04 계산 결과 I12의 지연 표시와 B4 기준일 확인

outputs/day4-document-automation/WBS_Gantt.xlsx

수정 대상: 복사본의 입력 셀 / 계산 셀은 유지

WBS의 최소 단위

항목	예시	용도
업무	PR diff 수집	끝낼 작업의 범위
Role	Backend	업무 담당 역할
시작·종료	09/16-09/18	기간 계산
선행 ID	W01	먼저 끝나야 할 업무
진척률	0.8 → 80%	현재 진행 정도

AI 초안과 계산의 분리

일	AI 도움	코드·수식
업무 분해	누락 후보와 순서 제안	필수 열·ID 검사
일정 산정	가정과 위험 설명	업무일 계산
상태 표시	지연 원인 문장 초안	날짜·진척률 조건
보고	읽기 쉬운 설명	같은 입력의 수치 재사용

업무일 계산

=NETWORKDAYS(D9,E9)

시작일과 종료일 포함

토요일·일요일 제외

공휴일 목록은 별도 추가

H열: 업무일

샘플: 대한민국 공휴일 미반영

셀 참조의 방향

참조	고정되는 부분	간트에서의 역할
L\$8	날짜가 있는 8행	가로 이동 시 다음 날짜
\$D9	시작일 D열	세로 이동 시 다음 업무
\$E9	종료일 E열	업무별 기간 비교
\$B\$4	기준일 셀 전체	모든 업무의 같은 기준일

간트 표시 조건

```
=AND(  
    L$8 >= $D9,  
    L$8 <= $E9,  
    WEEKDAY(L$8,2) < 6  
)
```

기간 안쪽 + 평일

실제 파일은 입력 검증·상태 조건도 포함

입력 열과 계산 열

	A	B	C	D	E	F
8	ID	업무	Role	시작일	종료일	진척률
9	W01	요구사항·완료 기준	PM	09/14	09/15	100%
10	W02	리뷰 정책·금지 행동	AI Engineer	09/16	09/17	100%
11	W03	PR diff 수집	Backend	09/16	09/18	100%
12	W04	Codex 리뷰 초안	AI Engineer	09/21	09/23	80%
13	W05	단위 테스트·실패 사례	QA	09/21	09/24	60%

입력 A:F 확대 · 전체 파일의 계산 H:K · 간트 L:AM

WBS 데이터 로딩

```
from labs.day4.office_lab.sheets.student_excel import (
    load_sample, calculate_wbs
)
tasks = load_sample("wbs_tasks.json")
rows = calculate_wbs(tasks, "2026-09-25")
```

Notebook 4차시

원본 JSON 대신 복사본 수정

업무 ID와 Python 행 선택

```
w04_index = next(
    i for i, row in enumerate(tasks) if row["id"] == "W04"
)
w04 = tasks[w04_index]
print(w04["id"], w04["task"])
print(w04["start"], w04["end"], w04["progress"])
```

Notebook 4차시의 데이터 로딩 셀 실행 후
W04: 2026-09-21 ~ 2026-09-23 / 진척률 0.8

진척률 변경

01 W04의 progress를 0.8에서 1로 변경

02 calculate_wbs 다시 실행

03 새 XLSX 복사본을 생성해 열기

wbs_tasks.json → 내 WBS 복사본

W04 지연 → 완료 · 완료 개수 변화

진척률 수정과 XLSX 저장

```
tasks_my = deepcopy(tasks)
tasks_my[w04_index]["progress"] = 1
destination = RUN / "excel/WBS_walkthrough.xlsx"
wbs_path = write_wbs_copy(
    ROOT, tasks_my, AS_OF,
    str(destination.relative_to(ROOT)),
)
print(wbs_path)
```

Notebook의 ROOT·RUN·AS_OF와 import 셀 선행
같은 이름이 있으면 WBS_walkthrough_v2.xlsx 사용

일정 변경과 차트 확인

01 새 복사본에서 B4 기준일 변경

02 D:E 시작·종료일 변경

03 I 상태와 L:AM 간트 이동 확인

WBS_Gantt.xlsx · 복사본

입력 변화와 차트 변화의 일치

입력 셀과 수식 입력줄

- 01 내 WBS를 열고 B4 기준일을 2026-09-25로 복구
- 02 D12=09/21·E12=09/23 복구, H12의 날짜 참조 확인
- 03 F12를 80%로 변경하고 I12의 지연 표시 확인
- 04 F12를 100%로 복구하고 완료 표시 확인 후 저장

내 실행 폴더 / excel/WBS_walkthrough.xlsx

기본 날짜 유지 시 H12=3 / 진척률에 따라 I12 변경

선행 일정의 오류

- 01 Python 복사본에서 W04 시작일을 2026-09-18로 변경
- 02 validate_wbs(broken_tasks)로 선행 일정 충돌 확인
- 03 Excel 복사본 D12도 09/18로 바꾸고 J12 확인
- 04 두 입력을 각각 복구하고 다시 확인

Notebook 4차시 · WBS J열

Excel 수정과 Python 변수 수정은 자동 연동되지 않음

종료일 오류와 입력 복구

```
broken_tasks = deepcopy(tasks)
broken_tasks[0]["end"] = "2026-09-01"
print(validate_wbs(broken_tasks))
broken_tasks[0]["end"] = tasks[0]["end"]
assert validate_wbs(broken_tasks) == []
print("날짜 복구 완료")
```

종료일이 시작일보다 빠르면 END_BEFORE_START
오류를 고친 뒤 검사 결과가 빈 목록인지 확인

Codex의 WBS 확장

대화 입력 예시

WBS 코드와 테스트를 읽어 줘. 공휴일 목록을 선택 입력으로 추가하고 싶어. 기존 기본 결과는 유지하고, 휴일이 있는 경우와 없는 경우의 테스트를 먼저 제안해 줘. 원본 XLSX는 덮어쓰지 마.

확인할 내용 기존 동작 · 추가 입력 · 테스트 두 종류

수식 결과의 검증

검증	방법
계산 정확성	작은 날짜 구간을 손으로 계산
재계산	입력 한 셀 변경 후 상태 확인
서식	같은 조건의 업무는 같은 표시
호환성	사용 중인 Excel·LibreOffice에서 열기

저장 후 값이 그대로인 경우

화면의 상태	확인과 복구
F12가 아직 80%	출력 경로 확인 새 저장본을 다시 열기
F12는 100%, I12는 지연	Excel 수식 탭 계산 옵션: 자동 / 지금 계산
Python에서 수식 결과가 None	openpyxl은 수식 미계산 Excel·LibreOffice에서 재계산
앱에서 계산 후에도 불일치	B4·D12·E12와 수식 확인 원본 템플릿 복사본으로 재현

4차시 확인

확인 항목	확인 방법
코드 수정	W04 진척률 변경과 새 파일 생성
동적 표	날짜 변경 후 상태·간트 갱신
잘못된 입력	선행 일정 충돌 확인
원본 보존	다른 이름의 XLSX 저장

WBS의 확장 기준

기능	먼저 필요한 정의
담당자별 부하	하루 가용 시간·병행 업무
비용 산정	단가·기간·수량의 단위
의존성 자동 조정	변경 전파와 예외 허용
휴일 처리	국가·조직·근무요일

템플릿과 생성 코드

안정된 레이아웃은 템플릿으로 유지

반복 입력과 계산은 코드로 분리

양식 변경도 버전과 테스트로 관리

4주차 5차시

13:50-14:40

자동 채점과 문항 분석

정답 변경·입력 오류·동점 순위의 일관된 처리

이론 12분 · 시연 8분 · 작업 25분 · 확인 5분

exam_sample.json · Exam_Grading.xlsx

40문항 자동 채점

8	정답				1	2	3	4	1	2	3	4
9	학습자 ID	비고	응시 상태		1	2	3	4	5	6	7	8
10	S001		응시 ▼		1	3	3	4	2	3	3	1
11	S002		응시 ▼		1	3	3	4	2	3	3	4
12	S003		응시 ▼		1	3	3	4	1	3	3	4
13	S004		응시 ▼		1	2	3	4	1	3	3	4
14	S005		응시 ▼		1	2	4	1	1	3	4	4

합성 응답 일부 확대 · 정답행과 1~8번 답안 · 실제 성적 미포함

채점 정책

상태	점수	집계
정상 응답	정답 1점·오답 0점	포함
미응답	해당 문항 0점	포함
잘못된 입력	점수 확정 보류	행 제외
결시	0점과 별도 관리	제외
정답 미등록	전체 채점 보류	정답 확인 후 재계산

채점 시트의 위치

위치	내용	변경 여부
D8:AQ8	40개 정답	교수자 입력
D10:AQ37	응시자 답안	학생 응답 입력
AR:AS	총점·동점 등수	수식 계산
AT:AX	보류·미응답·오류 집계	검증 결과
43행	문항별 정답률	분모를 확인할 지표

정답과 답안의 셀 선택

- 01 제공 채점 파일의 복사본에서 채점 탭 선택
- 02 이름 상자에 D8 입력: 첫 문항의 정답 1 확인
- 03 D10 선택: S001의 첫 문항 답안과 정답 비교
- 04 AR10의 총점 20점과 AT10의 처리 상태 확인

outputs/day4-document-automation/Exam_Grading.xlsx

정답 8행 / 답안 10~37행 / 총점 AR열

정답 색의 조건

```
=AND(  
    $C10="응시", D$45="등록",  
    ISNUMBER(D10),  
    D10=D$8,  
    D10>=1, D10<=4  
)
```

적용 범위 D10:AQ37

유효한 정답 등록 + 같은 열의 8행과 비교

동점 순위와 정답률

지표	규칙	예시
동점 순위	나보다 높은 점수 인원 +1	30·30·28 → 1·1·3
정답률 분자	집계 대상 중 해당 문항 정답자	정답자 수
정답률 분모	입력 오류 없는 응시자	미응답 포함·결시 제외
분모 0	집계 보류	0%로 오해 방지

총점과 보류 상태

	AR	AS	AT	AU	AV	AW	AX
9	총점 /40	등수	처리 상태	미응답	입력 오류	오답	정답

10-32행 생략 · 아래는 33-37행

	AR	AS	AT	AU	AV	AW	AX
33	32	1 채점 완료		0	0	8	32
34	19	26	미응답 포함	2	0	19	19
35	채점 보류	순위 제외	입력 오류	0	1	16	23
36	20	19 채점 완료		0	0	20	20
37	채점 보류	순위 제외	결시	0	0	0	0

합성 학생 24-28번 · 총점·등수·미응답·입력 오류 비교

Python 기준값

```
from labs.day4.office_lab.sheets.student_excel import (  
    load_sample, grade_exam  
)  
exam = load_sample("exam_sample.json")  
report = grade_exam(exam)
```

Excel 결과와 독립적으로 비교
기본 집계 대상 26명

문항 번호와 목록 위치

```
print("1번 정답:", exam["key"][0])  
student = exam["students"][25]  
print("학생:", student["id"])  
print("9번 답안:", student["answers"][8])  
print(grade_exam(exam)["students"][25])
```

26번째 학생 S026 / 9번 답안 9 / Excel L35
목록 위치는 0부터, 문항 번호는 1부터 시작

정답 변경의 영향

- 01 원본을 deepcopy로 복사
- 02 첫 문항 정답을 1에서 2로 변경
- 03 Python 보고서와 새 XLSX의 총점 비교

Notebook 5차시 · D8

총점·순위·정답률의 동시 갱신

입력 오류의 복구

- 01 L35의 9 입력 확인
- 02 1~4의 유효한 답안으로 수정
- 03 집계 대상 인원과 평균 재확인

Exam_Grading.xlsx · L35

집계 대상 26 → 27명

보기 번호 수정의 전후

```
exam_my = deepcopy(exam)
exam_my["students"][25]["answers"][8] = 1
before_count = grade_exam(exam)["eligible_count"]
fixed = grade_exam(exam_my)
print(before_count, fixed["eligible_count"])
print(fixed["students"][25]["score"])
```

집계 대상 26명에서 27명 / S026은 24점

수정한 exam_my를 Notebook의 XLSX·CSV 저장 셀에 전달

예외 입력 테스트

01 정답 한 칸을 비워 채점 오류 확인

02 답안에 글자 또는 9 입력

03 정답·답안을 복구하고 재계산

Notebook 5차시 · 별도 복사본

오류는 오류 · 복구 후 정상 계산

정답 누락의 복구 순서

```
missing = deepcopy(exam_my)
missing["key"][0] = None
held = grade_exam(missing)
assert held["students"][0]["score"] is None
print(held["students"][0]["status"])
missing["key"][0] = exam["key"][0]
assert grade_exam(missing)["eligible_count"] == 27
```

D8에 해당하는 정답 누락: 정답 확인 / 점수 보류
정답 복구 후, 입력 오류를 고친 27명 다시 집계

문항 분석

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
40	문항 분석			1	2	3	4	5	6	7	8	9	10	11	12	13
41	정답자 수			20	14	19	17	17	12	19	16	15	17	19	14	19
42	집계 대상			26	26	26	26	26	26	26	26	26	26	26	26	26
43	정답률			77%	54%	73%	65%	65%	46%	73%	62%	58%	65%	73%	54%	73%
44	미응답 수			0	0	0	1	0	0	0	0	0	0	1	0	0
45	정답 확인			등록	등록	등록	등록	등록	등록	등록	등록	등록	등록	등록	등록	등록
46																
47																
48	정답률 분모: 입력 오류가 없는 응시자 26명. 결시·입력 오류 행 제외, 미응답은 분모에 포함하며 0점 처리.															
49	동점 등수: 같은 점수는 같은 등수, 다음 등수는 인원수만큼 건너뛴. 결시·오류·정답 미등록은 총점과 등수 미확정.															
50	실습: 정답 D8 변경 → 총점·동점 순위·정답률 재계산. L35의 9를 1~4로 수정 → 집계 대상 증가.															

정답률만이 아닌 정답자 수·집계 대상·미응답 수

Google Sheets 적용

순서	내 연습 파일에서의 작업
1	합성 XLSX 업로드 → Google Sheets 사본
2	빈 문서는 제공 Code.gs로 합성 시트 생성
3	D10:AQ37 → 서식 → 조건부서식
4	맞춤 수식과 범위 확인
5	정답·오답·빈칸을 바꿔 실제 결과 비교

내 채점표의 Google Sheets 사본

- 01 개인 Drive의 새로 만들기에서 합성 XLSX 파일 업로드
- 02 업로드한 파일 열기, 파일 메뉴의 Google Sheets로 저장
- 03 새 Google 파일 이름을 4주차_채점_연습으로 변경
- 04 원래 XLSX 보존, 새 파일의 채점 탭에서만 작업

내 복사본의 Exam_Grading.xlsx / 개인 Google 계정

선택 확장 / 온라인 계산·서식은 계정에서 직접 확인 필요

조건부서식 메뉴와 시험 입력

- 01 채점 탭에서 응답 범위 D10:AQ37 선택
- 02 서식 메뉴의 조건부서식 열기, 맞춤 수식 규칙 선택
- 03 정답 규칙의 적용 범위와 D\$8·D\$45 참조 확인
- 04 L35를 9에서 1로 변경, B5=27과 AR35=24 확인

새 Google 파일 / 채점 탭 / 서식 > 조건부서식

원격 실행 미검증 / 계산과 색을 함께 확인 후 사용

채점 규칙 개선 요청

대화 입력 예시

합성 채점 코드에 대해 입력 오류, 미응답, 결시, 동점 처리 규칙을 표로 설명해 줘.
정답을 추측해서 바꾸지 마. 문항별 배점을 추가한다면 기존 1점 기본값을
유지하고 관련 테스트부터 제안해 줘.

확인할 내용

점수 정책 · 기존 결과 · 추가 테스트

5차시 확인

확인 항목	확인 방법
정답 변경	총점·순위·정답률 갱신
오류 처리	잘못된 답안은 채점 보류
분모	결시·입력 오류 제외 규칙
결과 파일	수정된 XLSX·CSV 열기

맞춤 수식의 공식 안내

The screenshot shows a Google Docs document titled "맞춤 수식의 공식 안내" (Custom Formula Guide). The document is in Korean and provides instructions on how to use custom formulas in Google Sheets. It includes a table of contents, a list of steps to create a custom formula, and examples of how to use them. The document is written in a clear, concise style with a light blue header and a white background. The text is in black, and there are some blue links and icons. The document is divided into sections by horizontal lines. The first section is titled "맞춤 수식이란?" (What is a custom formula?). The second section is titled "맞춤 수식 만들기" (How to create a custom formula?). The third section is titled "맞춤 수식 사용하기" (How to use a custom formula?). The fourth section is titled "맞춤 수식 예시" (Custom formula examples?). The document also includes a "참고" (Reference) section and a "로그인" (Login) button in the top right corner.

Google Docs 편집기 고객센터

문제를 설명해 주세요.

로그인

클릭하거나 새 규칙 추가 > 맞춤 수식:을 클릭합니다.

5. 값 또는 수식을 클릭하고 수식 및 규칙을 추가합니다.

6. 완료를 클릭합니다.

참고: 수식은 표준 표기법(=sheetname!cell)을 사용하여 동일한 시트만 참조할 수 있습니다. 수식에서 다른 시트를 참조하려면 INDIRECT 함수를 사용하세요.

예 1

데이터에 동일한 값이 2회 이상 나타나는 경우 강조표시하는 방법은 다음과 같습니다.

1. 컴퓨터를 사용하여 Google 스프레드시트에서 스프레드시트를 엽니다.
2. 서식을 지정할 범위를 선택합니다. 예를 들어 셀 A1부터 A100을 선택합니다.
3. 서식 > 조건부 서식을 클릭합니다.
4. '다음 경우 셀 서식 지정:' 드롭다운 메뉴에서 맞춤 수식:을 클릭합니다. 이미 규칙이 있는 경우 규칙을 클릭하거나 새 규칙 추가 > 맞춤 수식:을 클릭합니다.
5. 첫 번째 행에 적용할 규칙을 작성합니다. 이 경우 규칙은 다음과 같을 것입니다.
'=COUNTIF(\$A\$1:\$A\$100,A1)>1.'
6. 기타 서식 속성을 선택합니다.
7. 완료를 클릭합니다.

예 2

한 행에 있는 셀 중 한 셀의 값을 기준으로 전체 행의 서식을 지정하는 방법은 다음과 같습니다.

1. 컴퓨터를 사용하여 Google 스프레드시트에서 스프레드시트를 엽니다.
2. 서식을 지정할 범위(예: 열 A:E)를 선택합니다.
3. 서식 > 조건부 서식을 클릭합니다.

Google 공식 도움말 · 실제 브라우저 캡처

Apps Script 확장

기능	안전한 범위
수식·조건부서식 일괄 배치	새 합성 연습 시트만 생성
반복 실행	기존 동일 이름 시트 덮어쓰기 금지
권한	현재 문서만 접근
검증	생성 후 직접 정답·오류 입력 변경

4·5차시 수업 종료

쉬는 시간

14:40-15:00

파일 저장 · 15시 수업 시작

4주차 6차시

15:00-15:50

Word 이력서와 사실 검증

경력 사실을 보존한 문장 개선과 Word 생성

이론 12분 · 시연 8분 · 작업 25분 · 확인 5분

career_facts.json · Resume_After.docx

Word 이력서 개선 전

AI 제품 개발 경력

저는 여러 산업에서 AI 업무를 해왔고, 현재 이커머스에서 Agent 관련 서비스의 PM 업무를 담당하고 있습니다. 과거에는 교육, 의료, 금융 분야에서 AI Engineer로 서비스를 만들고 운영했습니다.

업무 경력

무신사 Agentic AI Side PM

제가 진행했던 업무는 쇼핑탐색 및 상품상세정보 Agent AID Chat, Voice Agent, AI 해설, SEO/GEO, 모두의 AI 서비스 담당 PM. 또한 AI 해설, SEO/GEO, 모두의 AI 서비스 담당 PM입니다.

개선 전 상단 확대 · 같은 경력 사실의 긴 문장·중복 표현

경력 문장의 구성

요소	질문	예시
상황	어떤 업무 문제였나	수작업 말하기·글쓰기 평가
Role	어떤 일을 맡았나	AI 개발·운영 총괄
실행	무엇을 만들었나	평가 시스템과 운영 흐름
결과	무엇이 달라졌나	처리시간 단축
근거	어떤 사실에 기반하나	career_facts.json의 ID

개선과 사실 변경

작업	허용	확인 필요
문장 정리	중복 제거·명사 통일	의미가 달라진 표현
성과 강조	제공한 수치 재사용	새 수치·비율
업무 범위	제공한 Role 설명	참여를 총괄로 변경
경력 구조	산업별 묶음	새 재직 기간·직함

Fact map

항목	용도
fact_id	문장을 원래 사실에 연결
원문	사용자가 제공한 경력
개선안	더 읽기 쉬운 설명
검사 결과	새 숫자·누락된 근거 탐지
사람 확인	의미·범위·톤앤매너 검토

경력 원문 JSON 열기

- 01 VS Code 탐색기에서 documents 폴더의 JSON 열기
- 02 파일 안에서 medical-polyp 검색
- 03 text의 원문과 rewrite의 수정 문장 비교
- 04 원본 저장 없이 Notebook의 my_rewrites에서 수정

labs/day4/office_lab/documents/career_facts.json

id는 근거 연결 / text는 원문 / rewrite는 수정 초안

Fact ID의 원문 조회

```
known = {  
    fact["id"]: fact  
    for role in facts["roles"] for fact in role["facts"]  
}  
source_fact = known["medical-polyp"]  
print(source_fact["text"])  
print(source_fact["rewrite"])
```

Notebook 6차시의 read_fact_map 셀 실행 후
문장 위치가 바뀌어도 같은 ID로 원문 조회

이력서 개선 요청

대화 입력 예시

career_facts.json의 사실만 사용해 Word 이력서를 개선해 줘. 새로운 숫자·직함·재직 기간은 만들지 마. 산업별 경험은 문제·Role·실행·결과 순서로 정리하고, 바뀐 문장마다 근거 fact_id를 함께 보여 줘.

확인할 내용

사실 연결 · 새 주장 여부 · 읽기 흐름

Word 이력서 개선 후

AI 제품 개발 경력

금융 머신러닝과 의료 딥러닝, 교육 LLM을 거쳐 이커머스 Agent 제품을 설계하고 운영합니다.

현재 Role

무신사 Agentic AI Side PM

- 쇼핑탐색 및 상품상세정보 Agent AID Chat과 Voice Agent 제품 기획 담당
- AI 해설, SEO/GEO, 모두의AI 서비스 기획 담당

개선 후 상단 확대 · 전체 내용은 편집 가능한 DOCX 제공

Word 생성 코드

```
python -m labs.day4.office_lab.documents.resume_lab \  
    --variant after \  
    --output outputs/day4-student/Resume_Mine.docx
```

python-docx · API key 불필요
출력 폴더는 작업 공간 내부

경력 문장 한 개 개선

- 01 Fact map에서 문장 하나 선택
- 02 의미는 유지하고 길이·순서 개선
- 03 audit_rewrite로 자동 검사

career_facts.json · Notebook 6차시

fact_id 유지 · 새 숫자 없음

문장 수정안의 자동 검사

```
my_rewrites = {  
    "medical-polyp":  
        "대장내시경 영상 기반 용종 탐지·진단 서비스 총괄 개발과 운영"  
}  
report = audit_rewrite(facts, my_rewrites)  
print(report["numeric_guard"])  
print(report["semantic_claims_verified"])
```

새 숫자 검사 PASS / 의미 검증 False
자동 검사 후에도 Role과 업무 범위는 원문 대조

의도한 과장 검사

01 복사한 개선안에 새 숫자 추가

02 자동 검사에서 차단 확인

03 숫자를 제거하고 다시 검사

Notebook 6차시 · audit_rewrite

새 숫자 차단 · 통과 후에도 사람 검토

숫자 없는 과장의 검사 한계

```
bad_proposal = {  
    "medical-polyp": "전 세계 모든 병원의 진단 서비스를 총괄"  
}  
limit = audit_rewrite(facts, bad_proposal)  
print(limit["numeric_guard"])  
print(limit["semantic_claims_verified"])
```

검사 한계용 허위 문장 / DOCX 저장 금지
숫자가 없어 PASS여도, 원문에 없는 주장

Word 스타일 수정

- 01 제목·본문·섹션 스타일 확인
- 02 글꼴 또는 단락 간격 한 항목 수정
- 03 DOCX 생성 후 Word/PDF 화면 비교

resume_lab.py → 내 DOCX

편집 가능한 본문 · 줄바꿈·페이지 확인

DOCX의 새 버전 생성

```
word_path = RUN / "word/Resume_walkthrough.docx"
result = build_resume(
    ROOT, FACT_SOURCE, word_path,
    variant="after", proposals=my_rewrites,
)
print(result["path"])
print(result["status"])
```

실제 파일 생성 / 상태는 REQUIRES_HUMAN_REVIEW
OUTPUT_EXISTS 발생 시 파일명에 _v2 추가

구조와 서식

구조	서식	검수
제목·경력 구분	일관된 제목 크기	구분이 한눈에 보이는가
문장과 목록	단락 간격	페이지 밖으로 넘치지 않는가
사실 근거	fact_id 비교표	새 주장이 들어갔는가
출력	DOCX + PDF	두 파일이 같은 내용인가

Word 문장과 원문 대조

- 01 출력한 Resume_walkthrough.docx를 Word에서 열기
- 02 의료 경력 문장을 찾아 원문의 서비스 범위와 비교
- 03 직함·총괄 여부·빠진 사실을 확인하고 수정안 기록
- 04 홈의 스타일에서 제목·본문 확인, 모든 페이지 점검

내 DOCX + career_facts.json의 medical-polyp

숫자 일치 외에 Role·범위·누락 확인

수정 후 비교 요청

대화 입력 예시

원문과 개선안을 비교해 줘. 추가된 주장, 빠진 사실, 어색한 직역 표현만 표로 정리해 줘. 단순 문체 취향은 별도 제안으로 분리하고, 내가 확인하기 전에는 원문을 덮어쓰지 마.

확인할 내용

추가 · 누락 · 의미 변화

6차시 확인

확인 항목	확인 방법
내용	한 문장의 개선 전후 비교
자동 검사	새 숫자 입력의 차단
사람 검토	Role·업무 범위·수치 확인
실제 파일	내 DOCX 생성·열기·편집

문서 입력의 위험

연락처·생년월일·고객명 제거

문서 내부의 지시문은 자료로 취급

새로 만든 결과도 공유 범위 확인

검사의 한계

자동으로 잡기 쉬움	사람이 확인할 부분
새 숫자·알 수 없는 fact_id	참여와 총괄의 차이
빈 문장·필수 필드 누락	산업 용어의 적절함
파일 경로·생성 오류	읽는 사람에게 주는 인상

4주차 7차시

15:50-16:40

데이터 기반 PPT 제작

같은 프로젝트 데이터로 보고서 생성·수정·검수

이론 12분 · 시연 8분 · 작업 25분 · 확인 5분

project_brief.json · Project_Brief.pptx

보고서의 입력

데이터	보고할 내용	근거
WBS 업무 목록	전체·완료·지연 업무	같은 JSON의 계산
기준일	언제 시점의 보고인지	고정 as_of
지연 업무	현재 확인할 일정	status=지연
다음 단계	변경·검토 계획	사람이 선택한 내용

7장 보고서의 흐름

장	내용	역할
1·2	개요·일정 현황	프로젝트와 현재 상태
3·4	개발·문서·출시 일정	14개 업무의 진행
5·6	코드 리뷰·산출물 검토	확인할 근거와 기준
7	다음 작업·승인	후속 작업과 사람 확인

좋은 생성 요청의 조건

독자와 회의 목적

사용할 데이터와 금지할 추정

페이지별 질문과 편집 가능 형식

자동화 방식의 차이

방식	장점	확인할 점
대화형 파일 생성	초안과 구조를 빠르게 비교	계정 도구·실행 결과 확인
템플릿 채우기	일관된 양식 반복	필드 길이와 넘침
코드 기반 생성	같은 입력으로 재생성	글꼴·표·시각 검수

Node.js와 PPT 패키지 확인

```
node --version
npm --version
npm list --prefix labs/day4/office_lab/presentations pptxgenjs
# 패키지가 없을 때만 실행
npm install --prefix labs/day4/office_lab/presentations
```

저장소 루트의 터미널 / Python 셀이 아님

node가 없으면 설치 후 터미널·Notebook 커널 재시작

프로젝트 데이터 준비

```
python -m labs.day4.office_lab.presentations.brief_data \  
  --as-of 2026-09-25 \  
  --output outputs/day4-student/brief.json
```

WBS와 같은 기준일
검증된 계산 → 보고서 입력

수정 WBS의 보고서 입력 저장

```
from labs.day4.office_lab.presentations.brief_data import (
    write_brief,
)
brief_path = write_brief(
    ROOT, str((RUN / "ppt/walkthrough.json").relative_to(ROOT)),
    tasks=tasks_my, as_of=AS_OF,
)
print(brief_path)
```

4차시의 tasks_my와 고정 기준일 AS_OF 재사용
같은 출력 이름이 있으면 walkthrough_v2.json 사용

학생용 PPT 생성

```
npm install --prefix labs/day4/office_lab/presentations  
node labs/day4/office_lab/presentations/student_ppt.mjs \  
  outputs/day4-student/brief.json \  
  outputs/day4-student/Project_Brief.pptx
```

저장소 루트에서 실행 · Node.js 필요
같은 파일명 재사용 시 새 이름 선택

Notebook의 PPT 생성 순서

- 01 Node.js 준비 셀 실행, 두 준비 상태 모두 확인
- 02 JSON 저장 셀 아래에 추가 코드 셀로 walkthrough.json 저장
- 03 생성 셀의 파일명을 Project_Brief_walkthrough.pptx로 변경
- 04 PPT 생성 셀 실행 후 출력 링크로 실제 파일 열기

Notebook 7차시 / student_ppt.mjs / 내 brief_path

PPT 생성 완료 메시지와 실제 7장 파일 확인

프로젝트 보고서 예시

일정 현황

전체 14개 업무 기준일 2026-09-25

상태	업무 수	확인 사항
완료	4	결과 파일과 검증 기록 확인
진행 중	2	마감일까지 잔여 작업 확인
지연	2	종료일 경과 및 미완료 업무
예정	6	선행 작업과 착수 조건 확인

Project_Brief.pptx · 편집 가능한 표와 수치

보고서 재생성

- 01 W04 진척률을 바꾼 tasks 준비
- 02 write_brief로 새 보고서 입력 생성
- 03 student_ppt.mjs로 새 PPT 생성

Notebook 7차시 · brief.json → PPTX

전체·완료·지연 수치의 변경 반영

완료·지연 수치의 입력 대조

```
data = json.loads(brief_path.read_text(encoding="utf-8"))
rows = calculate_wbs(tasks_my, AS_0F)
counts = {r[0]: int(r[1]) for r in data["slides"][1]["table"][1:]}
for status in ("완료", "지연"):
    expected = sum(r["status"] == status for r in rows)
    assert counts[status] == expected
assert data["wbs_snapshot"] == tasks_my
print(counts)
```

W04만 완료 처리한 입력: 완료 5개 / 지연 1개
서로 다른 기준일·입력 파일이면 비교 중단

편집 가능한 표 확인

01 생성 PPT를 PowerPoint/LibreOffice로 열기

02 표의 업무명 셀 직접 수정

03 본문과 표가 이미지가 아닌지 확인

내 Project_Brief.pptx

문자와 표 셀의 직접 편집

지연 업무와 보고 문장 비교

- 01 새 PPT 2장의 일정 현황에서 지연 업무 1개 확인
- 02 3장의 W04는 완료, W05는 지연인지 비교
- 03 7장의 후속 작업에서 W05의 지연 원인 확인 문장 검토
- 04 표와 문장이 다르면 입력·생성 코드 확인 후 재생성

내 PPT 2·3·7장 / W04 완료 입력 기준

업무 수가 같아도 후속 작업 문장은 별도 확인

Codex의 레이아웃 수정

대화 입력 예시

student_ppt.mjs를 읽고 본문 글씨를 더 크게 조정해 줘. 수치와 문장은 brief.json을 유지해 줘. 표를 이미지로 바꾸지 말고, 긴 업무명이 넘칠 때는 행 높이 또는 열 너비를 조정해 줘. 수정 전후를 렌더해서 비교해 줘.

확인할 내용

내용 유지 · 글자 크기 · 실제 화면

긴 업무명과 표 너비 조정

- 01 같은 폴더에 student_ppt_my.mjs라는 코드 복사본 생성
- 02 Codex에 긴 업무명 행의 colW·rowH 조정 요청
- 03 Notebook 실행 파일명을 복사본으로 바꾸고 새 PPT 생성
- 04 모든 7장에서 14개 업무의 누락·줄 잘림 확인

labs/day4/office_lab/presentations/student_ppt_my.mjs

표 편집 유지 / 글씨 축소보다 열 너비·행 높이 조정

PDF와 화면 검수

01 PPT를 PDF로 내보내기 또는 인쇄 저장

02 모든 페이지의 글꼴·잘림 확인

03 한 오류를 수정하고 다시 생성

PPTX + PDF · 내 출력 폴더

같은 내용 · 읽을 수 있는 글꼴

숫자·레이아웃 검증

검사	비교 대상
전체 업무	WBS 데이터의 행 수
완료·지연	동일 as_of의 계산 결과
긴 제목	줄바꿈과 표 높이
글꼴	다른 PC와 PDF 출력
파일 열기	PPTX 실제 편집 가능 여부

7차시 확인

확인 항목	확인 방법
코드 실행	새 입력으로 새 PPT 생성
데이터 연결	WBS와 보고서의 숫자 일치
편집 가능	본문·표 셀 직접 수정
화면 검수	PDF의 잘림·글꼴 확인

반복 보고의 구조

분리할 것	예시
Data	프로젝트·업무·기준일
Content	독자·핵심 질문·설명
Style	글꼴·색·간격·표
Validation	숫자·필수 페이지·렌더

배포 전 확인

개인정보와 비공개 프로젝트명 제거

버전·기준일·출처의 표시

메일·Drive 업로드는 별도 사람 확인

4주차 8차시

16:40-17:30

업무 도구 통합과 개인 프로젝트

반복 실행 가능한 패키지와 다음 주 제작 범위

이론 10분 · 시연 10분 · 작업 25분 · 확인 5분

README.md · improvement.md · 실행 결과 폴더

업무 자동화의 공통 구조

단계	이번 예제	확인 기준
입력	PR·WBS·답안·경력	범위와 필수 값
처리	리뷰·계산·문장 개선	역할별 함수
검증	테스트·수식·사실 비교	기대한 변화와 오류
출력	HTML·XLSX·DOCX·PPTX	실제 열기·편집
전달	개인 저장·선택 공유	사람이 대상 확인

서비스와 일회성 파일

구분	일회성 생성	반복 가능한 도구
입력	대화마다 다시 설명	정해진 필드·양식
실행	작성자만 방법을 앎	실행 명령·Notebook
오류	대충 수정 후 진행	메시지·복구 방법
검증	한 번 눈으로 확인	테스트 + 사람 확인
전달	파일만 공유	코드·안내·샘플 함께

Agent가 필요한 지점

고정된 계산과 생성은 코드

자료 탐색과 수정 제안은 AI

필요한 도구 선택·반복·확인을 묶는 작업 흐름

완성 패키지

파일	다른 사용자가 이해할 내용
README.md	시작 명령·입력·결과
requirements / package	설치할 라이브러리
sample + code	안전한 연습 데이터와 구현
tests	지켜야 할 정상·오류 규칙
improvement.md	수정 전후와 남은 제한

필수 검증 실행

```
python -m pytest -q \  
    tests/test_day4_pr_review_lab.py \  
    tests/test_day4_excel_lab.py \  
    tests/test_day4_document_lab.py
```

네트워크·유료 API 없는 기본 검증
학생 수정 코드는 해당 실습 폴더에서 별도 검사

Notebook의 결과 파일

- PR 리뷰 화면
- 리뷰 Markdown
- CI 설정 초안
- 내 WBS Excel
- 내 채점 Excel
- 채점 CSV
- 정답 변경 비교 Excel

생성 파일 목록 일부 확대 · 전체 결과는 index.html에서 확인

다음 주 3시간 제작

시간	개인 작업	작업 범위
15:00-15:20	문제·입력·완료 기준	기능 하나 선택
15:20-16:00	기본 기능 구현	샘플로 실제 실행
16:00-16:40	오류·테스트·개선	수정 전후 비교
16:40-17:10	화면·파일·사용 안내	다른 사람이 실행
17:10-17:30	패키지·개선 기록	미완료와 한계 구분
17:30-18:00	쉬는 시간·Q&A	선택 공유·실습 복구

제공 코드의 프로젝트 출발점

주제	최소 기능	확인할 개선
WBS 도우미	날짜·상태·간트	공휴일·선행 업무
채점 도구	점수·오류·통계	배점·입력 검증
이력서 편집기	사실 기반 DOCX	문장 비교·과장 검사
PR 리뷰 작업대	diff·테스트·리뷰	근거·중복·버전
보고서 생성기	같은 데이터의 PPT	수치 일치·긴 문장

PM · 프로젝트 관리

프로젝트	입력과 결과	출발 상태
PM01 일정·지연 점검	업무 JSON → WBS Excel	기본 코드 준비
PM02 주간 보고서	같은 WBS → 보고 PPT	기본 코드 준비
PM03 변경 요청 대장	요청 CSV → 변경 목록	새 로직 구현
PM04 위험 점검표	위험 CSV → 검토 HTML	새 로직 구현

고객지원 · 운영

프로젝트	입력과 결과	출발 상태
CS01 문의 분류	익명 문의 → 분류 CSV	새 로직 구현
CS02 FAQ 답변 초안	질문·FAQ → 근거 있는 MD	새 로직 구현
CS03 상담 인계	상담 기록 → 인계 DOCX	새 로직 구현
CS04 VOC 집계	문의 목록 → 유형별 표	새 로직 구현

영업 · 제안

프로젝트	입력과 결과	출발 상태
SA01 후속 연락 점검	활동 CSV → 예정 목록	새 로직 구현
SA02 견적 초안	승인 단가 → 견적 DOCX	새 로직 구현
SA03 미팅 후속 초안	합의 사항 → 메일 MD	새 로직 구현
SA04 제안 조건 비교	조건 CSV → 비교 Excel	새 로직 구현

콘텐츠 · 마케팅

프로젝트	입력과 결과	출발 상태
MK01 콘텐츠 일정	게시 계획 → 일정 Excel	기본 코드 응용
MK02 캠페인 URL 검사	링크 CSV → 오류 목록	새 로직 구현
MK03 광고 문구 검토	문구·근거 → 확인 목록	새 로직 구현
MK04 성과 보고	지표 CSV → 요약 PPT	출력 코드 응용

HR · 개인 경력

프로젝트	입력과 결과	출발 상태
HR01 경력 문서	경력 사실 → 이력서 Word	기본 코드 준비
HR02 포트폴리오 목록	프로젝트 CSV → 소개 HTML	새 로직 구현
HR03 온보딩 안내	업무 목록 → 체크리스트	기본 코드 응용
HR04 학습 계획	자가 점검 → 학습 목록	새 로직 구현

교육 · 학습 운영

프로젝트	입력과 결과	출발 상태
ED01 객관식 채점	합성 답안 → 채점 Excel	기본 코드 준비
ED02 문항 형식 검사	문항 JSON → 오류 목록	새 로직 구현
ED03 학습 피드백	교사 확인 항목 → Word	출력 코드 응용
ED04 출결 집계	합성 출결 → 집계 Excel	새 로직 구현

경영지원 · 문서

프로젝트	입력과 결과	출발 상태
AD01 지출 자료 점검	지출 CSV → 누락 목록	새 로직 구현
AD02 업무 인수인계	담당 업무 → 인계 Word	출력 코드 응용
AD03 문서 갱신 점검	기한 CSV → 예정 목록	새 로직 구현
AD04 구매 조건 비교	제품 조건 → 비교 Excel	새 로직 구현

개발 · 운영

프로젝트	입력과 결과	출발 상태
DV01 PR 리뷰 작업대	합성 diff → 리뷰 HTML	기본 코드 준비
DV02 테스트 실패 분류	pytest 로그 → 오류 MD	새 로직 구현
DV03 릴리스 점검	상태 JSON → 점검 HTML	새 로직 구현
DV04 실행 이력 보고	로그 CSV → 실패 집계	새 로직 구현

지식 · 회의

프로젝트	입력과 결과	출발 상태
KN01 회의 할 일	회의 TXT → 할 일 목록	2주차 코드 응용
KN02 결정 변경 비교	신·구 기록 → 차이 MD	새 로직 구현
KN03 자료 찾아보기	허용 MD → 검색 HTML	새 로직 구현
KN04 회의 사전 브리프	지정 자료 → 근거 목록	새 로직 구현

개인 과제의 범위

- 01 다음 주 만들고 싶은 도구 하나 선택
- 02 사용자·입력·결과를 README에 작성
- 03 정상 사례 1개·실패 사례 2개 정의

materials/day5/미니프로젝트_사전안내.md

무엇이 실행되면 완료인지 명확한 문장

사용자 한 명의 반복 업무

PM 민지: 매주 WBS에서 지연 업무 확인

입력: 업무 14개와 이번 기준일

첫 결과: 확인이 필요한 업무의 로컬 목록

첫 버전과 이후 확장

구분	첫 버전	나중의 개선
입력	지정한 파일 한 개	여러 시스템 수집
판단	날짜·상태의 명시적 규칙	AI의 원인 설명
결과	새 로컬 CSV 한 개	메일·Slack 전달
확인	정상 1건·실패 2건	다중 사용자·운영 로그

입력 파일의 세 가지 상태

파일	준비 내용	확인할 차이
sample_normal.json	형식이 맞는 업무 목록	기본 결과 생성
sample_bad_date.json	끝 날짜가 시작보다 빠름	날짜 오류 거절
sample_bad_id.json	존재하지 않는 선행 ID	관계 오류 거절

첫 제작 요청

대화 입력 예시

wbs_tasks.json과 student_excel.py를 읽어 줘. PM이 이번 기준일의 지연 업무만 보는 로컬 목록이 필요해. 입력 파일은 보존하고, calculate_wbs의 결과를 필터링하는 작은 함수를 제안해 줘. 파일 생성과 실행 전에 변경 위치·정상 사례·실패 사례 두 가지를 먼저 보여 줘. 외부 연결과 게시 기능은 제외해 줘.

확인할 내용

사용자 · 입력 경로 · 최소 함수 · 금지 행동

계획과 구현 결과의 구분

대화 응답	아직 확인할 것	다음 요청
구현 계획만 제시	실제 코드 변경 여부	계획한 함수의 코드 작성
코드 블록 제시	저장 위치·호출 방법	파일 저장과 실행 명령
테스트 통과 주장	실제 명령과 출력	정상·오류 결과 제시
CSV 생성 완료	파일 내용·열 구조	직접 열어 입력과 대조

개선 기능의 첫 구현

- 01 이번 코드에서 작은 기능 한 가지 선택
- 02 Codex에 함수·테스트의 수정 계획 요청
- 03 코드를 수정하고 해당 테스트 실행

내 개인 연습 폴더

실제 변경 diff와 실행 결과

기존 계산의 재사용

```
from labs.day4.office_lab.sheets.student_excel import (
    load_sample, calculate_wbs
)
def delayed_tasks(tasks, as_of):
    rows = calculate_wbs(tasks, as_of)
    return [row for row in rows if row["status"] == "지연"]
items = delayed_tasks(load_sample("wbs_tasks.json"),
                      "2026-09-25")
print([row["id"] for row in items])
```

기본 입력: 지연 업무 2개

새 함수는 계산을 다시 만들지 않고 결과를 선택

직접 확인할 세 가지 결과

입력	기대 결과	확인 방법
제공 업무·09/25	W04·W05 선택	ID와 원본 행 비교
종료일 역전	입력 오류로 중단	오류 코드와 위치 확인
없는 선행 ID	입력 오류로 중단	해당 ID 복구 후 재실행

파일 저장과 재실행 요청

대화 입력 예시

확인한 delayed_tasks 결과를 새 CSV로 저장해 줘. 입력 파일의 작업 폴더 밖 접근과 기존 출력 파일 덮어쓰기는 막아 줘. 정상 입력, 날짜 오류, 없는 선행 ID를 테스트하고 실제 명령과 결과를 남겨 줘. README에는 설치·입력·실행·결과 경로만 먼저 적어 줘. 내가 CSV를 열어 확인한 뒤 다음 기능을 정할게.

확인할 내용

실제 CSV · 테스트 출력 · 새 폴더 재실행

실행 안내의 검증

01 README의 명령을 그대로 복사

02 새 출력 폴더에 결과 생성

03 다른 파일을 덮지 않는지 확인

README.md + 새 출력 폴더

설명 없이 명령만으로 재실행 가능

개선 기록

항목	작성 내용
Before	기존의 불편과 재현 입력
Change	바꾼 함수·수식·문장
Evidence	테스트 출력·결과 화면
After	확인한 변화
Limit	아직 못한 일과 사용 조건

하루의 최종 확인

확인 항목	확인 방법
PR	코드 수정·테스트·게시 미리보기
Excel	입력 변경·재계산·오류 복구
Word·PPT	실제 파일 생성·내용·화면 확인
다음 주	한 기능의 개인 과제와 테스트 초안

작은 도구의 운영

상황	다음 개선
입력 형식 변경	버전 표시·변환 함수
다른 PC에서 실패	설치·폰트·경로 검사
반복 오류	에러 코드·재현 샘플
여러 명 사용	권한·동시 실행·개인정보
배포 요청	사람 확인·되돌리기 방법

추가 학습 경로

GitHub: 테스트와 리뷰의 자동 실행

Google Sheets: 수식·Apps Script·입력 검증

문서 도구: 구조·스타일·렌더 검증

4주차 수업 종료

쉬는 시간 · Q&A

17:30-18:00

질문 · 실행 오류 복구 · 다음 주 개인 과제